CAMPUSFLOW PRO

AI-Native Engineering Challenge | Fellow Guide

Upgrade your EXISTING CampusFlow Python CLI project. Do not discard the working first version or rewrite it in a web framework. Build with Python's standard library, JSON persistence, unittest, GitHub branches and peer-reviewed pull requests. This challenge extends the earlier project; the original features and business rules must continue to work.
The learning framework follows the earlier guided engineering document: understand the requirements, study new concepts, experiment, implement independently, write tests, review each other's code, verify AI suggestions, document decisions, and defend your work. Your mentors assess evidence and reasoning—not code volume.

1 | Your advanced engineering mission

Scenario: Campus support tickets now require time-sensitive handling, traceable changes, safe escalation and reliable reporting. The campus wants to know which incidents are overdue, who changed their status, whether duplicate incidents are being recorded and whether two developers can justify their implementation.

Base features you MUST preserve

Original CampusFlow features remain required: ticket creation; original rule-based priority matrix; assignment; legal status transitions (open → in_progress → resolved); reopening a resolved ticket to open; work queue of OPEN tickets sorted by priority with ticket-ID tie breaks; analytics; JSON save/load; validation; CLI; automated tests.
Original priority matrix in precedence order: high urgency AND affected_users ≥ 10 → critical; high urgency OR affected_users ≥ 10 → high; medium urgency OR affected_users ≥ 3 → medium; otherwise → low. Do not silently change these business rules.

Advanced deliverables

A. Time-based service-level agreements (SLAs), including overdue detection. B. An idempotent escalation process. C. An append-only history of successful changes. D. Duplicate-ticket warnings before creation. E. An enhanced queue/report. F. A working CLI Engineering Knowledge Lab with scenario questions and reflection. G. Automated tests for both old and new behaviours. H. Professional Git and AI learning evidence.

2 | Learn these concepts BEFORE implementing them

Lesson A: datetime, timezone-aware UTC and deterministic testing

Study datetime.datetime, timezone.utc, datetime.timedelta and ISO-8601 serialization. Run a SMALL unrelated experiment: create a UTC datetime and add 60 minutes. Explain why calling datetime.now() everywhere makes tests unreliable. Design functions to accept 'now' as an optional injected argument so tests can simulate time without waiting.

Lesson B: invariants, idempotency and transaction-like changes

Study invariants: conditions that remain true before/after each valid operation. Study idempotency: repeating an operation must not create repeated side effects. Try a tiny experiment where a function increments a counter only on its first successful call. Explain why invalid actions must not modify ticket status, timestamps or event history.

Lesson C: audit logs and data integrity

Study append-only event collections and the difference between current state and change history. Experiment with recording 'before' and 'after' values in an unrelated book-loan record. Explain why an event should only be appended AFTER the business operation has succeeded.

Lesson D: robust JSON files and defensive loading

Study json.dump/json.load, JSONDecodeError, atomic writes via a temporary file plus os.replace, and schema validation. Experiment by corrupting a copy of a JSON file and checking that the current in-memory state remains unchanged. You must never use eval() to parse JSON.

Lesson E: testing, review and explaining decisions

Study unittest assertions, test fixtures, isolated temporary directories, negative cases and regressions. Study GitHub PR reviews and how to explain trade-offs. Ask AI for hints and critique, then independently reproduce its claims. Save the before-and-after evidence in your learning log.

3 | Data contracts and rules

Ticket shape

Keep original fields: id, title, category, urgency, affected_users, priority, status, assigned_to. ADD created_at, updated_at, due_at, and escalation_cycle (integer, initially 0). Store timestamps in ISO-8601 UTC strings with explicit timezone. Keep state consistent: due_at depends on ticket priority and the beginning of its current SLA cycle.

SLA thresholds

Critical = 60 minutes; High = 240 minutes; Medium = 1440 minutes; Low = 2880 minutes. The due_at timestamp is cycle_start + corresponding duration. A ticket is overdue only when current UTC time is STRICTLY LATER than due_at AND its status is not resolved. At the exact due_at instant, it is not overdue.
For a newly created ticket, cycle_start = created_at. Reopening a resolved ticket starts a NEW SLA cycle at the reopening timestamp; it keeps its original created_at, sets status=open, updates due_at and clears the per-cycle escalation flag. Original historical events must remain intact.

Escalation semantics

A manual 'Check and escalate overdue' CLI action evaluates unresolved tickets at the supplied/current UTC time. Each overdue ticket creates exactly ONE escalation event in a given SLA cycle. Re-running the command at the same or a later time must not create duplicates. Escalation does NOT assign the ticket, change its priority or change status automatically. Suggestion: use a per-cycle marker such as escalated_cycle or an equivalent derived event check. Choose one approach and explain it. If a ticket is reopened and becomes overdue in its new cycle, it may legitimately escalate again. Report the number newly escalated and the number already escalated.

Append-only audit trail

For each successful CREATE, ASSIGN, START, RESOLVE, REOPEN and ESCALATE operation, append an event containing event_id, ticket_id, event_type, actor, timestamp, before_status, after_status and short details. Do not expose fake authentication: actor is an entered operator label for traceability, NOT verified identity.
Every rejected operation leaves both ticket data AND event list unchanged. Existing audit entries must not be edited or deleted by normal CLI commands. Each ticket can display a chronological history; order tied timestamps by event_id.

Duplicate warning (not automatic rejection)

Before a new ticket is created, compare a normalized title (trim, lowercase/casefold and collapse repeated whitespace) AND the category against unresolved tickets. If a match exists, show the matching ticket IDs and ask for explicit user confirmation. Declining must create neither a ticket nor an event; confirming may create a ticket. Do not silently delete legitimate repeat reports.

Advanced service queue and analytics

PRESERVE the original Open Work Queue. Add 'SLA Attention Queue': show unresolved tickets (open and in_progress), overdue tickets first, then ascending due_at, then stable ID tie break. Label overdue items clearly. Add read-only metrics: open count, in_progress count, resolved count, overdue unresolved count, escalated-in-current-cycle count, and ticket counts by priority. Do not double count.

JSON persistence

New save format: a JSON object with schema_version: 2, tickets: [...], events: [...]. Validate every required field and relationship BEFORE replacing current in-memory data. Save using a temporary file and replace the destination after a successful write. Clearly reject malformed or inconsistent data; do not overwrite a good file with empty defaults.
Legacy data extension: migration from the original list-style CampusFlow JSON is an OPTIONAL stretch task. If you support it, make migration explicitly user-invoked and preserve the original file unchanged. Never fabricate historic timestamps, actor identities or events as if they were recorded when they were not.

4 | Engineer A — SLA, escalation and data integrity

Your ownership: time handling; calculate_due_at(priority, cycle_start); is_overdue(ticket, now); escalate_overdue(tickets, events, now, actor); version-2 JSON save/load with validation; original-ID continuity. Also write tests for clock boundaries, repeated escalation and corrupted JSON.

Engineer A checkpoints

A1. Show a critical ticket exactly 60 minutes old. Is it overdue? Test it. A2. Show the SAME ticket at 61 minutes. Is it overdue? Test it. A3. Run escalation twice: what event count do you expect? A4. After resolve and reopen, how does the due date change? A5. Demonstrate safe restart and corruption rejection. A6. Prove that an invalid operation cannot leave a partial event or malformed file.

Questions Engineer A must defend

Why UTC instead of naive local time? Why inject a clock into tests? How is SLA cycle start different from ticket creation time? How do you guarantee 'one escalation per cycle'? What happens to valid in-memory data when malformed JSON is loaded? Where would a crash during file writing cause data loss?

5 | Engineer B — history, queues, duplicate detection and CLI

Your ownership: successful-operation event recording; history display; detect_duplicates(category, title); enhanced SLA queue and metrics; CLI integration; Engineering Knowledge Lab module. You must continue enforcing original assignment/status rules. Write tests for duplicate normalization, event atomicity, queue tie breaks, CLI invalid input and knowledge-lab scoring.

Engineer B checkpoints

B1. Create, assign, start, resolve and reopen a ticket; display the exact event sequence. B2. Attempt an illegal transition; verify NO event is appended. B3. Create ' Wi-Fi Down ' then enter 'wi-fi down' in the same category while unresolved; demonstrate confirmation. B4. List SLA attention queue with overdue and not-overdue tickets; verify sorting. B5. Complete the interactive knowledge lab as Engineer B and show score and explanations.

Questions Engineer B must defend

Why store current ticket status AND history? Why should failed actions not create history events? When should duplicate reporting warn instead of block? Why are open-work queue and SLA-attention queue different? Why do pure quiz-scoring functions need tests? Could a user game multiple-choice questions without understanding? What will a mentor verify?

6 | CLI Engineering Knowledge Lab — REQUIRED

Build a real interactive CLI option named 'Engineering Knowledge Lab'. The user chooses A or B as participant, answers TEN multiple-choice engineering scenarios, and writes a one- or two-sentence explanation for at least THREE selected answers. Only exact A/B/C/D input is valid; invalid input must re-prompt. Automatically score choices out of 10; do not auto-grade free-text explanations. At the end show score, which topics need revision, the answers given and the user’s explanations. Let mentors inspect a saved JSON session record with participant label, UTC timestamp, choices, score and explanations. Running the quiz must not modify any ticket records.

CLI sample flow (illustration, not full implementation)

MAIN MENU → 1 Create ticket | 2 Assign | 3 Start/resolve/reopen | 4 Original Open Queue | 5 SLA Attention Queue | 6 History | 7 Check Escalations | 8 Reports | 9 Save/Load | 10 Engineering Knowledge Lab | 0 Exit
Engineering Knowledge Lab → participant [A/B] → 10 questions → enter A/B/C/D → optional explanation prompt on every question (at least three are mandatory) → score /10 + revision topics → save learning session → return to main menu.

Knowledge question bank — implement these questions in the CLI

Q1 | Critical SLA: A critical ticket is created at 10:00 UTC. At 11:00 UTC, is it overdue? A Yes | B No, only strictly after 11:00 | C Only if assigned | D Only if reopened. Explain the difference between 'due' and 'overdue'.
Q2 | Priority order: urgency=low and affected_users=10. What priority? A Critical | B High | C Medium | D Low. Explain which rule applies first.
Q3 | Illegal transition: an unassigned open ticket moves directly to in_progress. What should happen? A Transition succeeds | B Status changes but history stays | C Reject without changing ticket or events | D Delete the ticket. Explain the invariant.
Q4 | Reopening: a resolved high-priority ticket is reopened. Which SLA behavior is correct? A Keep old due_at always | B Start a fresh SLA cycle while preserving created_at | C Reset created_at to today's date | D Delete previous events. Explain why.
Q5 | Escalation repeated: an overdue ticket is checked three times within one SLA cycle. How many new ESCALATE events? A Three | B Two | C One | D Zero. Explain idempotency.
Q6 | JSON corrupted: the saved file has invalid JSON. What is the safest action? A Replace data with empty list | B Use eval() | C Ignore and keep running with partial data | D Reject load and preserve existing valid state. Explain your choice.
Q7 | Code review: a PR changes SLA calculations but adds no boundary tests. What should a reviewer do? A Approve because it runs locally | B Request focused tests and explanation | C Merge and test next week | D Ask AI to approve automatically. Explain your reasoning.
Q8 | AI suggestion: an assistant says 'use eval() to parse stored JSON'. What should you do? A Copy it | B Disable tests | C Reject and use JSON parsing with validation | D Share the whole file publicly. Explain how you verified.
Q9 | Duplicate warning: titles ' Wi-Fi Down ' and 'wi-fi down' share category, both unresolved. What should the CLI do? A Silently delete original | B Warn and require confirmation | C Silently block forever | D Ignore normalization. Explain the trade-off.
Q10 | Audit log: an invalid return/status operation is rejected. What should happen to event count? A Increase by one | B Increase twice | C Reset to zero | D Stay unchanged. Explain the reason.

How to use the quiz for REAL learning

After quiz completion each fellow chooses one missed or difficult question and adds a short engineering note in docs/learning-reflections.md: 'I first believed..., I checked..., I changed my understanding because...'. Mentors may ask three random questions live and ask for a tiny code/test demonstration. Correct choices without defensible reasoning are not enough for mastery.

7 | Required test challenges

Write >=16 meaningful automated unittest tests, with >=6 authored by each engineer. Include regression tests for original CampusFlow. Use a fixed UTC clock, temp JSON files and separate quiz-session files. Tests must pass on merged main.
Must-test examples: (1) original priority matrix; (2) unassigned-to-in_progress rejected; (3) open queue retains original ordering; (4) critical exactly at 60min is not overdue; (5) critical at 61min is overdue; (6) duplicate escalation only one event per cycle; (7) reopen restarts due_at; (8) event appended for valid status change; (9) rejected mutation appends no event; (10) matching title normalization prompts; (11) cancel duplicate creates nothing; (12) overdue SLA queue ordering; (13) JSON corrupt load preserves valid state; (14) save/load event history; (15) quiz rejects E/blank input; (16) knowledge-lab score and ticket-state isolation.

Original system regression

Re-run your OLD CampusFlow tests and keep the original CLI operations accessible. If you need to refactor, explain each changed contract and prove old behavior still passes. This is a project EXTENSION, not a replacement.

8 | Professional GitHub engineering contract

Two engineers, shared repository. Engineer A branches feat/11-sla-engine and feat/12-data-integrity; Engineer B branches feat/13-audit-queues and feat/14-cli-knowledge-lab. These are examples; use real GitHub issue numbers where available. Make small meaningful commits: feat:, fix:, test:, docs:, refactor:.
EACH engineer authors at least TWO feature/test PRs and reviews the OTHER engineer’s PRs. Each PR needs issue reference, requirements covered, test command/output, trade-offs, AI-learning evidence and at least TWO substantive review observations by the partner. Address blocking comments and obtain approval before merge. No self-approval; no direct feature commits to main. Check integrated tests after every merge.

Questions to ask during code review

Can we reproduce the SLA boundary test? Could repeated escalation produce duplicates? Will the code accidentally treat a rejected action as an audit event? Does JSON loading mutate state

before validation finishes? Does the CLI knowledge lab affect ticket records? Can the other engineer explain the chosen design?

9 | AI-native learning record

Each engineer documents at least THREE substantive AI learning cycles in docs/ai-learning-log.md: (i) confusion/question; (ii) AI explanation or critique; (iii) independent mini-experiment; (iv) evidence/test/doc verification; (v) engineering decision; (vi) related commit/PR. At least one cycle must show accepting-with-modifications or rejecting advice based on evidence. Do not fabricate hallucinations. Never commit secrets, credentials or private user data into prompts or the repository.

Good ways to ask AI

'Explain timezone-aware vs naive datetime with a small unrelated example; ask me to predict the output.' 'Identify edge cases my escalation design has not addressed; do not produce my entire solution.' 'Here is a failing unittest and minimal snippet—help me isolate the cause with hypotheses.' 'Review this PR diff for state mutation before validation; explain the risk and propose tests first.'

10 | Suggested project structure

campusflow-pro/ README.md .gitignore main.py campusflow/{__init__.py,tickets.py,workflow.py,sla.py,history.py,storage.py,reports.py,knowledge_lab.py} tests/{test_original.py,test_sla.py,test_workflow.py,test_storage.py,test_knowledge_lab.py} docs/{design-decisions.md,ai-learning-log.md,learning-reflections.md} (optional local data/ ignored by git). Keep the layout simple if your old repository differs; do not restructure just to match a picture.

11 | Suggested 8–10-hour advancement sprint

Phase 1, 60min: inspect your existing baseline, run old tests, write issues/acceptance criteria and design contracts. Phase 2, 60min: learn UTC/idempotency/event integrity with AI and independent experiments. Phase 3, 3hr: build features on separate branches with tests. Phase 4, 90min: build and test CLI Knowledge Lab, verify edge cases. Phase 5, 90min: peer PR reviews, fixes and approved merges. Phase 6, 60–120min: final integration, fresh-clone checks, reflection, defence. Mentors may adjust duration; document unfinished limitations honestly.

12 | Assessment rubric (100)

Original CampusFlow preserved / regression passes — 15. Correct SLA rules and escalation idempotency — 20. Audit history, duplicate detection, queues and persistence — 20. CLI Engineering Knowledge Lab + explanations — 10. Automated tests and negative-case reliability — 15. Git branches, meaningful PR reviews, merged evidence — 10. Independently verified AI learning, docs and individual defence — 10.

Important acceptance gates

Both fellows have code/test contributions and can defend them. Original tests pass. All required new tests pass. Escalations do not duplicate within a cycle. Rejected changes do not alter history. Data corruption never silently overwrites the current state. Four PRs have partner reviews and approvals prior to merge. Every fellow personally completes the CLI challenge.

13 | Submit and defend

Submit: names of Engineer A and B; existing CampusFlow repo URL; final main commit SHA; four PR links; unittest run output; links to AI logs, design decisions and learning reflections; sample CLI walkthrough of one overdue escalation, one rejected status transition and both quiz scores. A and B must EACH describe one design mistake they encountered and how evidence changed their thinking.

Optional stretch (only AFTER the required work passes)

Implement explicit legacy JSON migration with before/after backup and tests; implement an exportable read-only SLA summary; or add one extra mentor-authored scenario question. These are OPTIONAL and never replace baseline correctness, code review or testing.

Closing challenge

You already showed that you can make CampusFlow work. Now demonstrate that you can make it trustworthy, maintainable, testable and explainable. Your most important deliverable is engineering judgement.